LAPORAN PRAKTIKUM

Pemrograman Berorientasi Objek

" Packages, Encapsulation, & Interfaces"



Nim	:	41425010
Nama	:	Apostel Mangara Simatupang
Prodi	:	DIV-Teknologi Rekayasa Perangkat Lunak

FAKULTAS VOKASI

INSTITUT TEKNOLOGI DEL

1. Lakukan analisa mengapa atribut jlhTorsi, faktorPembagi, panjangGagang, dan rasioGigi termasuk setter-getter nya pada kelas Motor dan Mobil tidak dibuat di dalam IKenderaan? Buatlah laporan anda dalam bentuk *softcopy*.
Jawab:

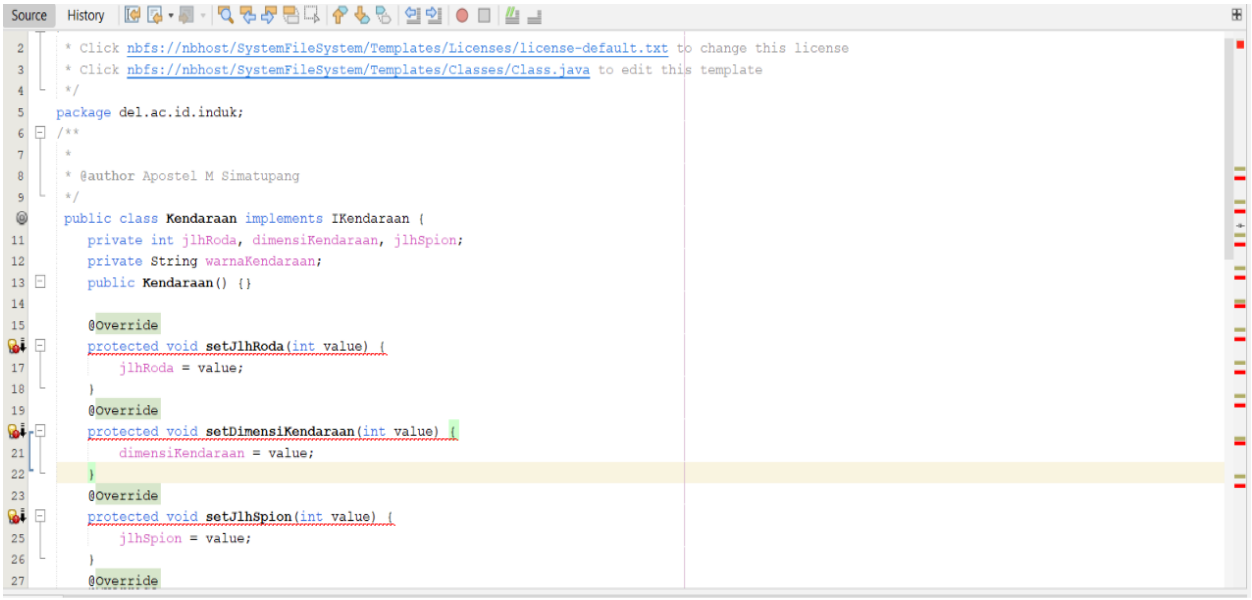
Atribut tersebut tidak dibuat di dalam IKendaraan karena interface hanya berisi aturan atau kemampuan umum yang harus dimiliki oleh kelas yang mengimplementasikannya. Interface IKendaraan digunakan untuk mendefinisikan method yang bersifat umum untuk kendaraan, sedangkan jlhTorsi, faktorPembagi, panjangGagang, dan rasioGigi merupakan atribut yang lebih khusus pada jenis kendaraan tertentu.

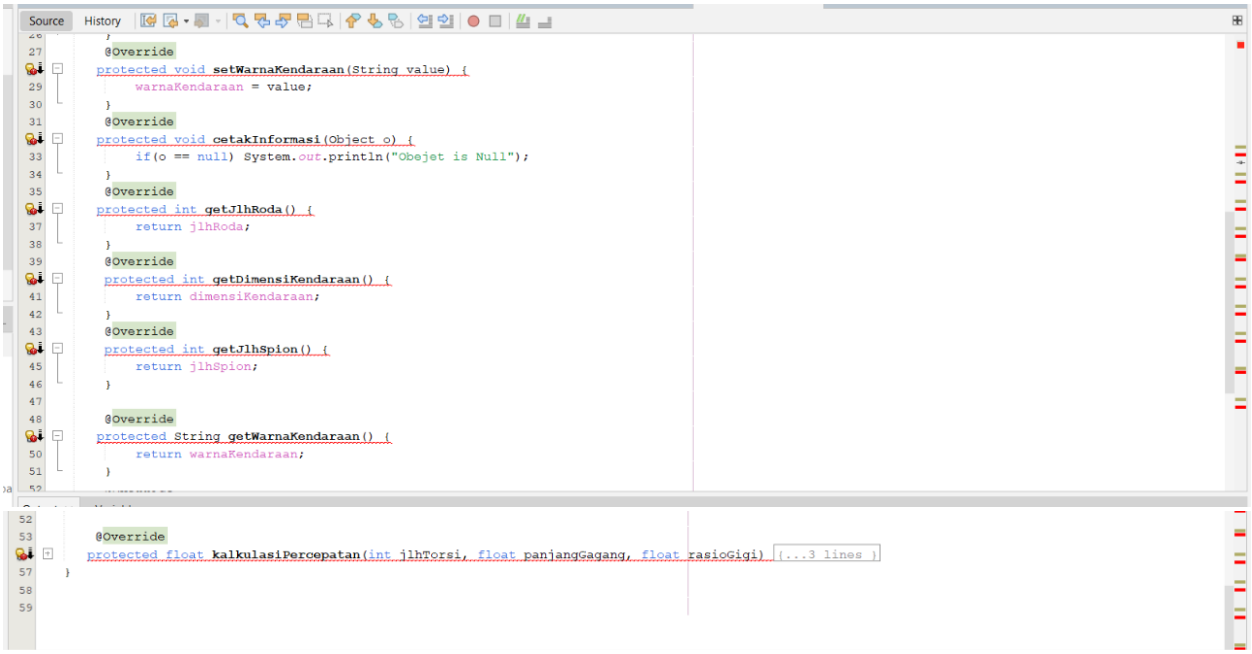
Contohnya, jlhTorsi lebih berkaitan dengan karakteristik kendaraan seperti motor atau mobil tertentu. Tidak semua kendaraan harus mempunyai atribut dan cara pengolahan data yang sama. Oleh karena itu, atribut tersebut lebih tepat diletakkan pada class yang membutuhkan, seperti Motor atau Mobil.

Kesimpulan:
IKendaraan digunakan untuk menentukan kemampuan umum kendaraan, sedangkan atribut khusus seperti jlhTorsi, faktorPembagi, panjangGagang, dan rasioGigi diletakkan pada class yang memang membutuhkannya.

2. Ubahlah seluruh *access modifier* metode yang ada pada kelas “Kendaraan” menjadi “protected”. Lakukan observasi kecil, mengapa perubahan ini mengakibatkan *error* pada program. Buatlah laporan anda dalam bentuk *softcopy*.

Jawab:





Penjelasan:

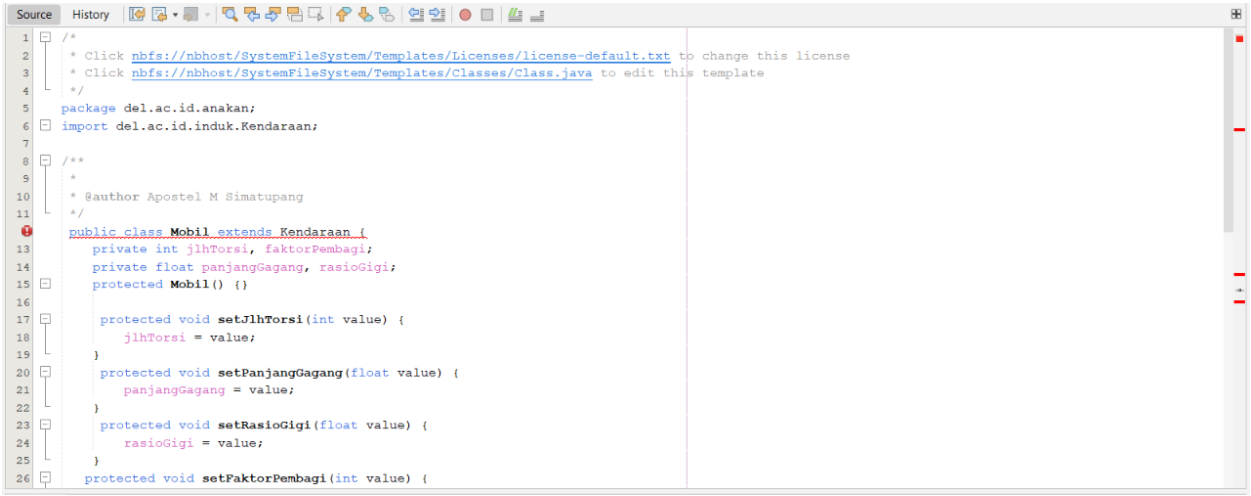
Pada kode tersebut terjadi error karena semua method pada class Kendaraan diubah access modifier-nya dari public menjadi protected. Sementara itu, method yang ada pada interface IKendaraan secara otomatis memiliki access modifier public. Ketika class Kendaraan mengimplementasikan interface tersebut, method yang dibuat untuk menggantikan method dari interface tidak boleh memiliki akses yang lebih kecil.

Jadi, ketika method pada Kendaraan menggunakan protected, Java menganggap akses method tersebut terlalu terbatas dibandingkan method pada IKendaraan yang bersifat public. Karena itu muncul error pada method yang menggunakan @Override. Error tersebut terjadi karena Kendaraan tidak boleh mengurangi tingkat akses method yang berasal dari interface.

Dengan kata sederhana, method pada IKendaraan adalah public, sehingga method yang mengimplementasikannya di Kendaraan juga harus public. Jika diubah menjadi protected, program akan error karena aksesnya menjadi lebih terbatas. Perubahan menjadi protected inilah yang menyebabkan error pada praktikum nomor 2.

3. Sama seperti soal no.2, lakukan perubahan *access modifier* metode pada kelas “Mobil” menjadi “protected”. Lakukan observasi kecil, mengapa perubahan ini mengakibatkan *error* pada program. Buatlah laporan anda dalam bentuk *softcopy*.

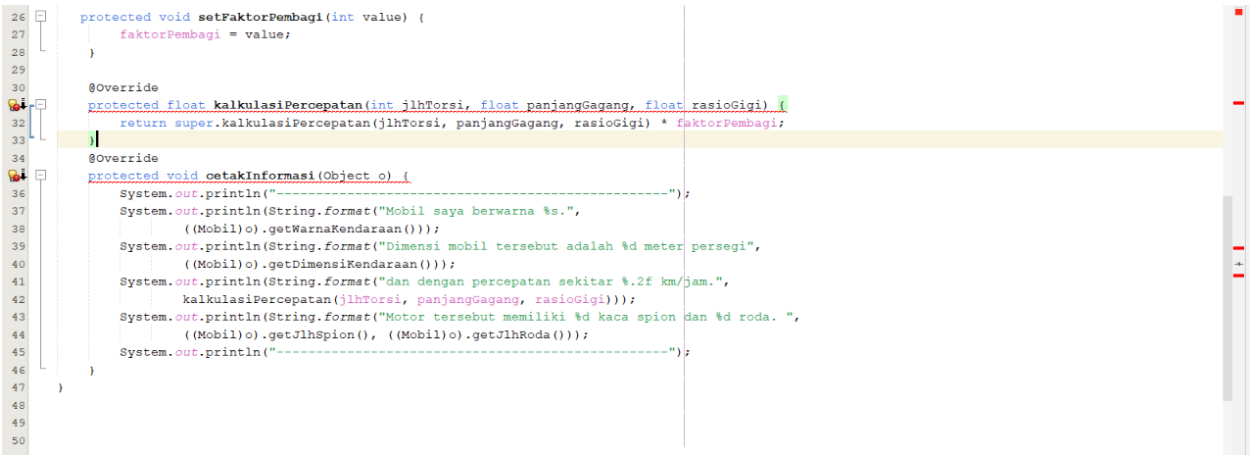
Jawab:



```
Source  History
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package del.ac.id.anakan;
import del.ac.id.induk.Kendaraan;

/**
 *
 * @author Apostel M Simatupang
 */
public class Mobil extends Kendaraan {
    private int jlhTorsi, faktorPembagi;
    private float panjangGagang, rasioGigi;
    protected Mobil() {}

    protected void setJlhTorsi(int value) {
        jlhTorsi = value;
    }
    protected void setPanjangGagang(float value) {
        panjangGagang = value;
    }
    protected void setRasioGigi(float value) {
        rasioGigi = value;
    }
    protected void setFaktorPembagi(int value) {
```



```
    protected void setFaktorPembagi(int value) {
        faktorPembagi = value;
    }

    @Override
    protected float kalkulasiPercepatan(int jlhTorsi, float panjangGagang, float rasioGigi) {
        return super.kalkulasiPercepatan(jlhTorsi, panjangGagang, rasioGigi) * faktorPembagi;
    }
    @Override
    protected void cetakInformasi(Object o) {
        System.out.println("-------------------------------------------------");
        System.out.println(String.format("Mobil saya berwarna %s.",
                ((Mobil)o).getWarnaKendaraan()));
        System.out.println(String.format("Dimensi mobil tersebut adalah %d meter persegi",
                ((Mobil)o).getDimensiKendaraan()));
        System.out.println(String.format("dan dengan percepatan sekitar %.2f km/jam.",
                kalkulasiPercepatan(jlhTorsi, panjangGagang, rasioGigi)));
        System.out.println(String.format("Motor tersebut memiliki %d kaca spion dan %d roda. ",
                ((Mobil)o).getJlhSpion(), ((Mobil)o).getJlhRoda()));
        System.out.println("-------------------------------------------------");
    }
}
```

Penjelasan:

Pada class Mobil, beberapa method seperti setJlhTorsi(), setPanjangGagang(), setRasioGigi(), setFaktorPembagi(), kalkulasiPercepatan(), dan cetakInformasi() menggunakan access modifier protected. Access modifier protected membuat method tersebut tidak dapat dipanggil secara bebas dari class utama yang berada di luar package.

Pada kode ini, class Mobil merupakan turunan dari class Kendaraan, sehingga Mobil masih dapat menggunakan method protected yang diwariskan dari Kendaraan. Namun, ketika method tersebut dipanggil dari class utama Percobaan_packages, aksesnya dapat menjadi masalah karena class utama bukan turunan dari Mobil dan berada di package yang berbeda.

Selain itu, method kalkulasiPercepatan() dan cetakInformasi() melakukan overriding terhadap method dari class Kendaraan. Jika method pada Kendaraan juga menggunakan protected, maka

penggunaan protected pada Mobil masih diperbolehkan karena tingkat aksesnya sama. Jadi, error pada praktikum nomor 3 terutama terjadi ketika method Mobil yang bersifat protected dipanggil langsung dari class utama.

Dengan kata sederhana, protected membatasi siapa yang boleh menggunakan sebuah method. Class Mobil dan class turunannya masih dapat menggunakan method tersebut, tetapi class lain yang berada di luar hubungan pewarisan tidak dapat memanggilnya secara langsung. Oleh karena itu, ketika program utama mencoba memanggil method Mobil yang sudah diubah menjadi protected, program dapat menghasilkan error.